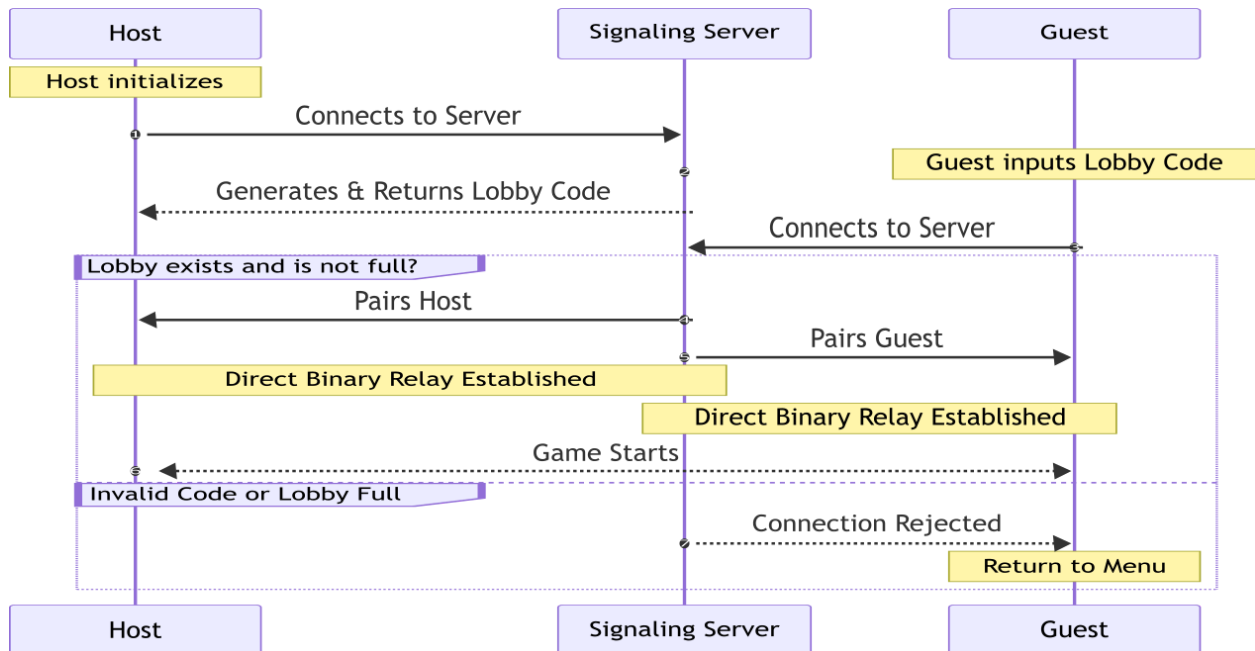


Architecture and Implementation of Schrödinger's Pong

Schrödinger's Pong is a real-time, cross-platform multiplayer adaptation of the classic arcade game, engineered from the ground up in C++. The primary objective was to solve a persistent challenge in networked game development: building a highly responsive system that minimizes the visual impact of latency while maintaining a strictly synchronized game state across disparate clients. To accomplish this without relying on off-the-shelf networking engines, the project implements a custom binary transport protocol over WebSockets, a Phase-Locked Loop clock synchronization algorithm, and deterministic fixed-point mathematics. Furthermore, it introduces a distributed-authority model with optimistic timeline prediction, allowing for instantaneous hit detection that elegantly masks network delays. Together, these systems mitigate network jitter and ensure a seamless competitive experience whether playing on a native desktop executable or a WebAssembly browser build.

Networking Architecture and Protocols

My architecture utilizes a Host-Guest topology brokered by a centralized signaling server. The Host acts as the authoritative simulation environment, while the Guest client receives game state updates and transmits its local paddle inputs.



All network communication is conducted over WebSockets. To overcome the overhead of standard WebSocket text frames, I implemented a highly compressed custom binary protocol.

State changes are serialized using Variable-Length Quantity (VLQ) and Zig-Zag encoding. Standard 32-bit integers rigidly consume 4 bytes of data, regardless of the value's size. VLQ optimizes this by using only the lower 7 bits of each byte for actual data, reserving the highest bit as a continuation flag. This allows the system to compress small positive numbers into a single byte.

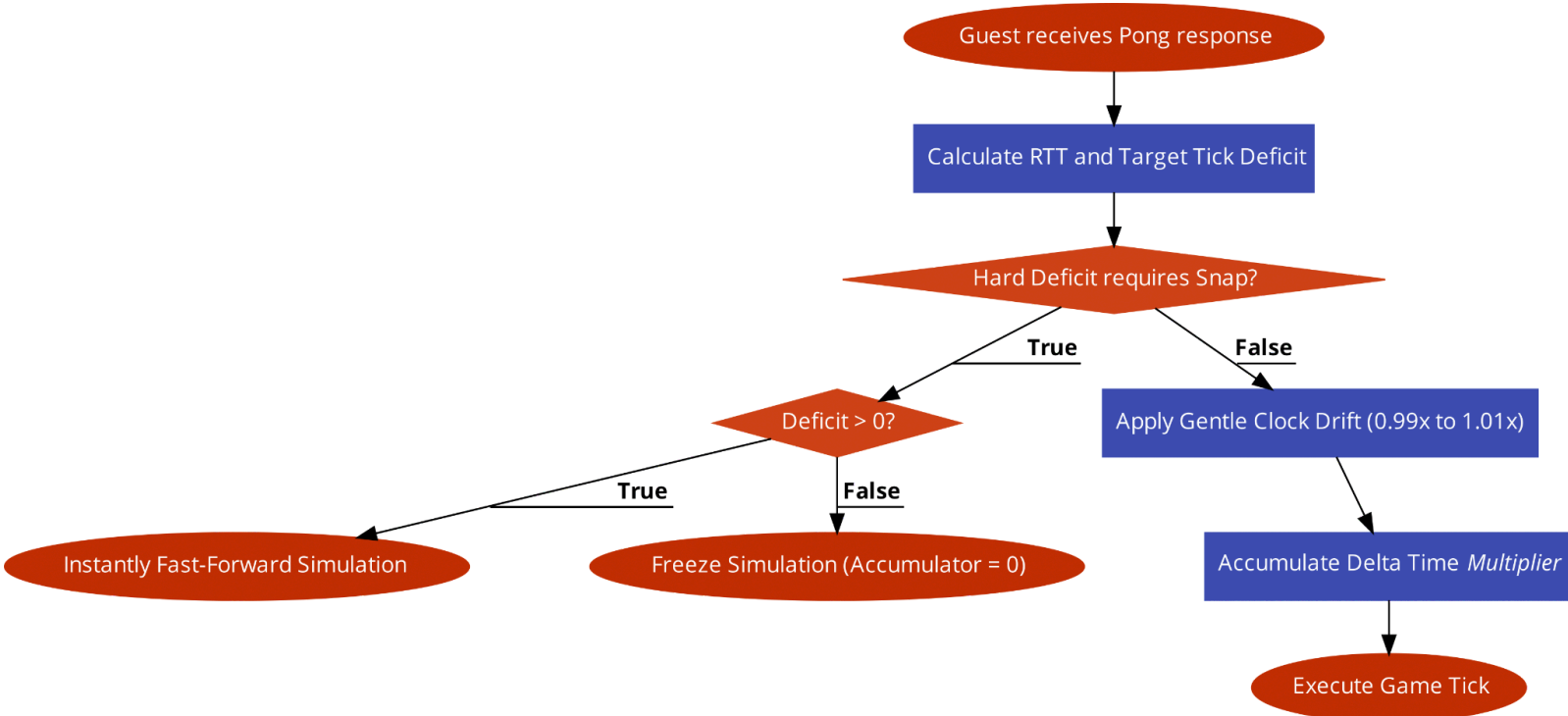
However, VLQ fundamentally struggles with negative numbers due to two's complement sign-extension. To circumvent this, the architecture applies Zig-Zag encoding before VLQ. This maps signed integers to unsigned integers by oscillating between positive and negative values (e.g., -1 becomes 1, 1 becomes 2). By mapping values with a small absolute value to small unsigned integers, negative positional deltas are compressed into a single byte.

Instead of broadcasting the entire game state every tick, the system calculates a delta bitmask to identify changed variables and transmits only those values. The result is that the maximum payload for a game state packet never exceeds 48 bytes.

Addressing Network Challenges

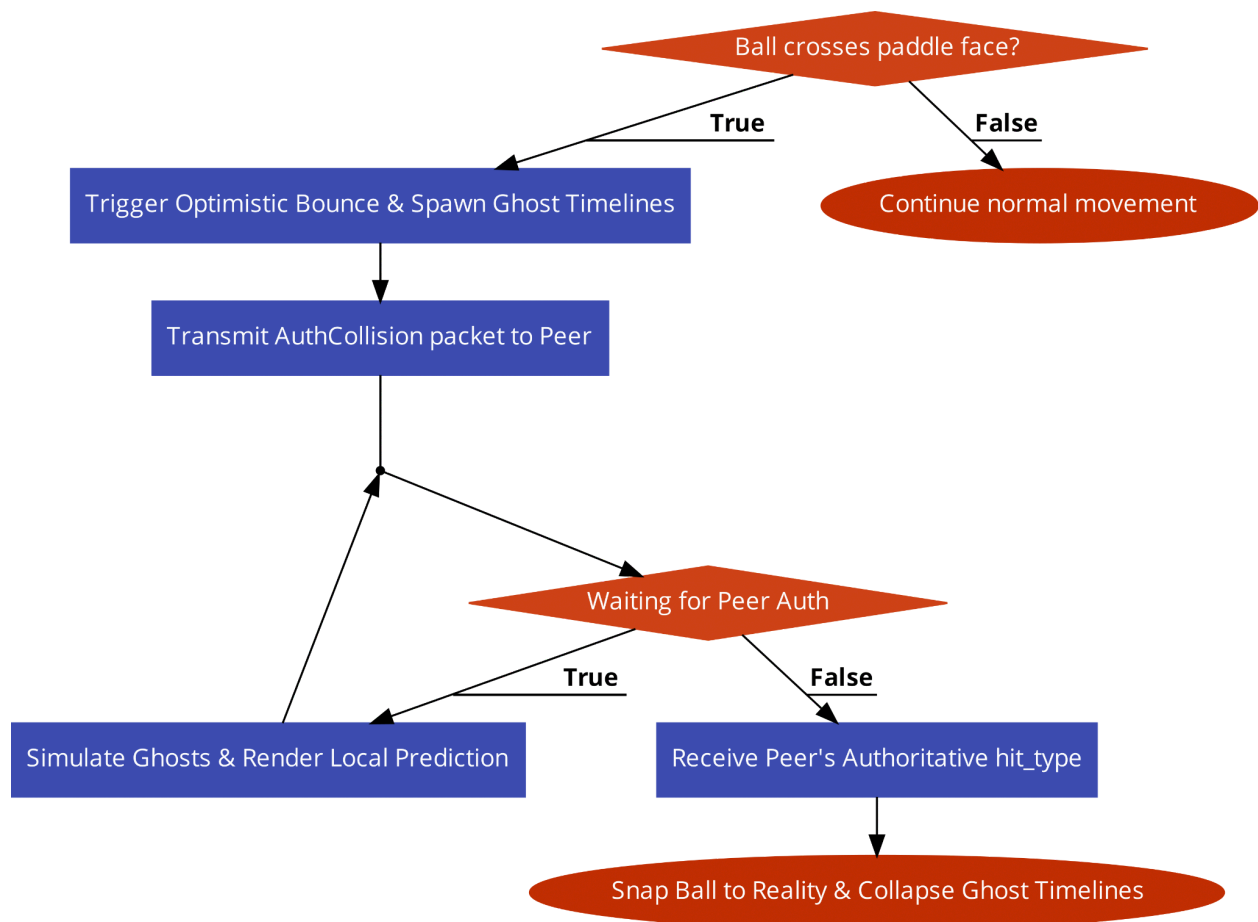
Latency Mitigation

To keep the Guest and Host synchronized, I engineered a Phase-Locked Loop tick-pacing algorithm. The Guest calculates a target simulation tick based on continuous Round Trip Time estimations and applies a clock drift multiplier; gently accelerating (1.01x) or decelerating (0.99x) the local simulation to catch up or fall back without jarring visual snaps.



Hiding Hit-Detection Latency

To mask the inherent delay in network transit, I designed the "Schrödinger Ball" mechanic. When the ball intersects a paddle, the local client optimistically predicts the bounce and visually updates immediately. The ball mathematically splits into multiple timelines until the peer confirms the collision with an authoritative message, at which point the false timelines are collapsed.



Reliability

Because the architecture relies on WebSockets for its binary relay, the underlying transport protocol is TCP. While TCP guarantees the delivery of packets, this creates a challenge

for real-time games: Head-of-Line (HoL) blocking. If a packet containing a critical game event is lost in transit, TCP will halt all subsequent data delivery to the client until the dropped packet is retransmitted and acknowledged.

To prevent these TCP retransmission cycles from causing massive latency spikes during gameplay, I implemented a redundancy system for critical one-shot events. When a player strikes the ball, the AuthCollision authorization is actively transmitted over 30 consecutive frames. This brute-force redundancy ensures that even amidst network packet loss, the peer receives the authoritative hit state immediately, bypassing the latency penalty of waiting for an OS-level TCP retransmission.

Furthermore, because TCP connections can be silently severed by NATs or firewalls during periods of inactivity, the architecture implements an application-layer heartbeat. The Host continuously transmits keepalive packets to the signaling server while waiting for a Guest to join, preventing the socket from timing out and ensuring connection stability across all platforms.

Scalability

The centralized C++ signaling server is only responsible for parsing lightweight JSON to broker the initial peer connections. Once a lobby is established, the server transitions into a binary relay, passing raw byte buffers directly between the two connected WebSockets without inspecting the payload. This drastically reduces server CPU load and allows for horizontal scaling.

Design Decisions and Tradeoffs

Determinism and Fixed-Point Mathematics

To keep a networked game synchronized, the simulation must be perfectly deterministic; meaning that if both clients are given the exact same inputs, they must produce the exact same outputs. While floating-point arithmetic (standard float or double variables) is the default way to calculate fractional movement, it is notoriously non-deterministic. Different CPU architectures, compilers, or browser runtimes (such as WebAssembly) handle floating-point rounding a bit differently at the deepest decimal levels. In a physics simulation running at 60 ticks per second, a rounding difference on frame one will snowball over time. Eventually, this causes a desync, where the ball might clip the edge of a paddle on the Host but completely miss on the Guest.

To prevent this, I strictly utilized fixed-point mathematics for the simulation. Fixed-point arithmetic eliminates floating-point decimals entirely, instead representing fractional values using standard integers scaled by a fixed, constant factor. In this architecture, all spatial coordinates are scaled by a factor of 100, meaning the game tracks objects in 1/100-pixel units. For example, a coordinate of 5.25 pixels is stored and calculated purely as the integer 525. Because basic integer arithmetic is handled identically at the binary level across all modern processors, this guarantees that both the Native C++ Host and the WebAssembly Guest will calculate the exact same mathematical trajectory, preventing the simulations from ever drifting out of sync.

Distributed Authority and Eliminating "Ghost Hits"

In a standard client-server networking model, the server is the absolute, centralized authority over the game state. While this prevents cheating, it introduces a frustrating issue for fast-paced games known as the "ghost hit" phenomenon. Because of network latency, a client's local rendering is always slightly behind the server's actual state. A player might see their paddle intercept the ball perfectly on their screen, but by the time their input reaches the server, the server calculates that the ball has already passed the paddle. The server rigidly rules it a miss, causing the ball to abruptly teleport past the paddle on the client's screen, essentially phasing right through it like a ghost.

To eliminate this issue and guarantee immediate player responsiveness, I inverted the standard model by distributing authority for hit detection. Instead of relying on a central arbiter, both the Host and the Guest are entirely authoritative over their own respective paddle faces. If a client registers a collision locally, it is treated as an absolute fact and transmitted to the peer. By trusting the local client, the system completely eradicates ghost hits and input lag. To seamlessly manage the slight timeline desynchronization this distributed trust creates, this architecture works in tandem with the Schrödinger rollback system to smoothly resolve the resulting state disputes.

Conclusion and Performance Results

Schrödinger's Pong demonstrates that the traditional limitations of real-time, web-based multiplayer can be effectively mitigated through aggressive latency compensation and deterministic engineering. By synthesizing a custom binary transport, Phase-Locked Loop clock pacing, and the distributed-authority timeline rollback system, the project achieves a seamless

cross-platform experience. As a final proof of concept, the architecture was stress-tested under severely degraded network conditions. Remarkably, the integrated systems successfully maintained a synchronized, playable game state even when subjected to a staggering 1000-millisecond Round Trip Time. This benchmark validates the architecture, proving that trusting the local client and optimistically predicting game state can effectively render even extreme network latency invisible to the end user.

References

- Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). Internet Engineering Task Force (IETF). Details the underlying bidirectional communication protocol utilized for the signaling and binary relay system.
- Fiedler, G. (2004). State Synchronization. Gaffer On Games. Explores the mechanics of delta compression, fixed-point math, and state synchronization in fast-paced networked games.
- Bernier, Y. W. (2001). Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization. Valve Corporation. Provides foundational theory on client-side prediction, clock synchronization, and hiding network latency from the end-user.
- Google (2008). Protocol Buffers Documentation: Encoding. Google Developers. Explains the mathematical implementation of Base 128 Varints and ZigZag encoding for efficient serialization of signed integers.